

Assignment 2 — Build a RAG System for Supply Chain Documents

Level: Beginner · **Effort:** 8–10 hours · **Submission:** GitHub repository link + 3-minute demo video

1. What you are building (in plain language)

A large language model like GPT-4o has never seen your company's internal procurement policy. If you ask it "what penalty applies when a supplier delivers late?", it will invent a plausible-sounding answer, because every company writes its own rules.

RAG (Retrieval Augmented Generation) solves this in five steps:

1. **Read** the PDFs and pull out their text.
2. **Chunk** — cut the text into small pieces of a few hundred words each.
3. **Embed** — convert each chunk into a list of numbers (a vector) that captures its meaning.
4. **Store** those vectors in a vector database (ChromaDB) so they can be searched by meaning rather than by exact keyword.
5. **Ask** — turn the user's question into a vector, pull the closest 4–5 chunks out of Chroma, and send *only those chunks plus the question* to GPT-4o, which answers using the text you supplied.

That is the whole system. Nothing more is needed here.

2. The business scenario

Meridian Components Pvt. Ltd. is an automotive electronics manufacturer. Two kinds of document govern its supply chain: a quarterly **performance review** full of numbers, and a **procurement policy handbook** full of rules. Today, when a supplier misses a delivery, the buyer has to read the review to find the numbers, then hunt through the policy to find which clause is triggered and what action is mandated. That takes twenty minutes and often gets done wrong.

Your job is to build an internal assistant where a buyer asks one question in plain English and gets the answer, drawn from whichever document holds it — or from both.

3. Data you will use

Two PDFs are provided with this assignment:

File	What it contains
Meridian_Supply_Chain_Review_Q1_FY2025-26.pdf	Supplier scorecards, freight lane costs, inventory metrics, line-stoppage incidents, quality data, risks, and planned actions
Meridian_Procurement_Policy_Handbook_v4.2.pdf	Supplier classification, approval limits, onboarding rules, scorecard weights, penalty clauses, safety-stock formulas, and escalation rules

You must index **both** documents into the same Chroma collection. The interesting questions in Section 7 can only be answered by combining a number from the first document with a rule from the second, which is exactly what this assignment is testing.

Optional extra: add your own third PDF (any supplier contract, logistics report, or incident note you write yourself) and confirm the system still works. This is a nice-to-have, not a requirement.

4. Technical requirements (all mandatory)

Component	What you must use
Language	Python 3.10 or above
PDF reading	pypdf , pdfplumber , or LangChain's PyPDFLoader
Chunking	Recursive character splitting, chunk size 800–1200 characters, overlap 100–200 characters
Embeddings	OpenAI text-embedding-3-small
Vector database	ChromaDB , persisted to a folder on disk
Answering model	GPT-4o , temperature between 0 and 0.2
Orchestration	LangChain or LangGraph or the plain <code>openai</code> SDK — your choice, all three are acceptable
User interface	Streamlit or Gradio
API key handling	Stored in a <code>.env</code> file, loaded with <code>python-dotenv</code> , and <code>.env</code> listed in <code>.gitignore</code>

5. Features your application must have

1. **Upload** — the user can upload one or more PDF files from the interface.
2. **Index** — a button that processes the uploaded files and reports something like "2 files processed, 96 chunks stored".
3. **Ask** — a text box for the question and a submit button.
4. **Answer** — the GPT-4o answer displayed clearly.
5. **Sources** — under every answer, show which document and page each retrieved chunk came from. A buyer must be able to open the handbook and confirm the clause before acting on it.
6. **Honest refusal** — when the answer is not in the documents, the app must say so rather than guess. Use a system prompt such as: "Answer only from the context provided below. If the context does not contain the answer, say the information is not available in the uploaded documents."
7. **Persistence** — close the app, reopen it, and the indexed documents must still be searchable without re-uploading.

6. Optional bonus — a FastAPI backend

For extra marks, move the logic into a FastAPI service exposing these endpoints:

Method	Endpoint	Input	Output
POST	/ingest	One or more PDF files	{"files": 2, "chunks": 96}
POST	/ask	{"question": "...", "top_k": 4}	{"answer": "...", "sources": [{"file": "...", "page": 3}]}
GET	/stats	—	Collection name, total chunks, embedding model, LLM model

The interface then calls these endpoints over HTTP instead of doing the work itself. Run it with `uvicorn main:app --reload` and check every endpoint from the automatic documentation page at `http://localhost:8000/docs`.

7. Test questions — your app must handle all of these

Questions 1 to 4 are answered from a single document. Questions 5 to 9 need both documents together — these are the ones that separate a working system from a lucky one. Record every answer in your README.

1. Which supplier had the highest spend in Q1, and what was its on-time delivery percentage?
2. How many line stoppages happened in Q1, what was the total downtime, and what caused them?
3. What is the approval authority for a purchase order worth ₹1.4 crore?
4. What are the four supplier classification categories, and what qualifies a supplier as Critical?
5. Kaveri Metals recorded 88.1% on-time delivery and 1,150 defects per million in Q1. Which policy clauses does this trigger, and what exactly must the buyer do?
6. The microcontroller supplier is single-source. What does the sourcing policy require in this situation, and what is the company already doing about it?
7. Microcontrollers are imported with a 46-day lead time. Using the safety-stock policy, how many days of stock should be held for this part?
8. Trident Circuit Boards had a defect rate of 640 parts per million. What is the cost consequence under the policy?
9. Which suppliers would fall below the B rating band on on-time delivery alone, and what is the escalation path for them?
10. **Deliberate trap question:** ask something the documents definitely do not contain, for example "What is the annual salary of the Head of Procurement?" Your app must reply that the information is not available rather than inventing a number. This question carries marks.

8. What you must submit

A public GitHub repository with the following structure:

```
supplychain-rag/
├── app.py           # Streamlit / Gradio interface
├── ingest.py        # load, chunk, embed, store in Chroma
├── rag.py           # retrieve + prompt + call GPT-4o
├── api/main.py      # optional FastAPI backend
├── data/            # the two provided PDFs
├── chroma_db/       # persisted vector store (add to .gitignore if large)
├── requirements.txt
├── .env.example     # OPENAI_API_KEY=your_key_here
├── .gitignore       # must include .env
└── README.md
```

Your `README.md` must contain: setup and run instructions a stranger can follow, your chunk size and overlap with a one-line reason for the choice, screenshots of the working app, all ten questions with the answers your app produced, and an honest note on which questions it got wrong and why you think that happened.

Also record a **3-minute video** showing upload, indexing, at least two cross-document questions being answered correctly, and the trap question being refused.

9. How you will be marked

Area	Marks
Ingestion pipeline — loading, chunking, embedding without errors	20
ChromaDB used correctly and survives a restart	15
Single-document questions answered correctly	15
Cross-document questions (5–9) answered correctly with both sources shown	20
Working user interface	10
Correct refusal on the trap question	10
Code quality, README, and repository hygiene	10
Bonus: FastAPI backend with all three endpoints working	+15

10. Rules, hints, and common mistakes

- **Keep it simple.** No re-ranking, no hybrid search, no agents, no fine-tuning. Anything beyond the requirements earns no extra marks and usually breaks something that was working.
- **Never put your API key in code.** Keys committed to GitHub are detected and disabled within minutes.
- **Cross-document questions usually fail because `top_k` is too small.** If the answer needs a number from the review and a clause from the handbook, and you only retrieve 3 chunks, all 3 may come from the same document. Try `top_k` of 5 or 6 and look at what actually came back.
- **Check retrieval before blaming the model.** Print the retrieved chunks for each failing question. Most wrong answers are caused by the wrong chunks being fetched, not by GPT-4o reasoning badly.
- **Tables lose their shape.** Scorecards and rate tables become messy text once extracted. A chunk size near 1200 characters often keeps a full table together and improves answers noticeably.
- **Verify by hand before submitting.** Open the PDFs and confirm at least three of your app's answers, including one penalty clause. Marks are for correct answers, not confident ones.

11. Submission checklist

☐ Both provided PDFs indexed into one Chroma collection ☐ Chunking works and reports chunk count ☐ Chroma folder persists after restart ☐ GPT-4o answers with document name and page number ☐ At least four of the five cross-document questions answered correctly ☐ Trap question refused correctly ☐ `.env` is in `.gitignore` ☐ README complete with screenshots and the ten answers ☐ 3-minute demo video recorded ☐ Repository link submitted